



ADITYA UNIVERSITY

Advanced Data Structures & Algorithm Analysis (241CS010)

UNIT-I

Prepared by
Anil Kumar Prathipati,
 Assistant Professor,
 Website/blog: anilkumarprathipati.wordpress.com
 ADITYA University

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 1



PREREQUISITE

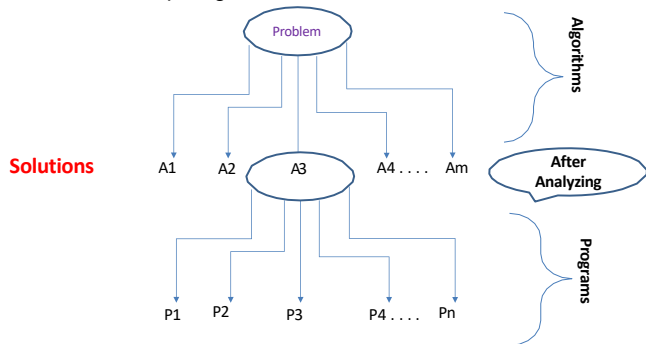
- Data Structures

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 2



Why Study this Course?

- Art of the Computing



Algorithm + Data Structure = Program

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 3



Contd ...

Example: Consider the Sorting Problem

Two Algorithms:

- 1) Naive based Algorithm (like brute force) $\implies O(n^2)$
- 2) Best Algorithm $\implies O(n \log n)$

Coming to System Performance (Speed)

CPU with speed (1 Ghz) 10^9 operations per second

Problem is to sort around billion (10^9) Phone numbers.

By using: Algorithm -1, What is the total time it takes?
 11,574 Days $\rightarrow$ 31.7 Yrs

Algorithm -2, What is the total time it takes?
 30 Seconds (App)

Q): Which Algorithm do you choose?

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 4

Contd ...

Example: Consider the Computer GAME⁵
 Let, If there are Five Lakhs (5×10^5) objects and all objects are moving (screen resolution $1920 * 1080$ more than 20 Lakhs pixels). If You are pointing at one object. If your game is based on the nearest path from your pointing object. Consider the previous CPU speed.

Two Algorithms:
 1) Navie based Algorithm (like brute force) $\Rightarrow O(n)^2$
 2) Best Algorithm $\Rightarrow O(n \log n)$

By using Algorithm -1, How much time does it takes?
 Algorithm -2, How much time does it takes?
Q): Which Algorithm do you choose?

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 5

UNIT-I SYLLABUS

Introduction to Algorithm Analysis, Space and Time Complexity analysis, Asymptotic Notations.

Hashing: Introduction to Static Hashing, Hash Tables, Hash Functions, Different Hash Functions, Collision Resolution Techniques, Dynamic Hashing.

[Complete SYLLABUS](#)
[Complete Lesson Plan](#)

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 6

1. What is an Algorithm?

- Name of a Persian mathematician Abu Jafar Mohammed Ibn Musa Al Khwarizmi (ninth century).
- Finite set of instructions that accomplishes a particular task.

or

- sequence of unambiguous instructions for solving a problem

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 7

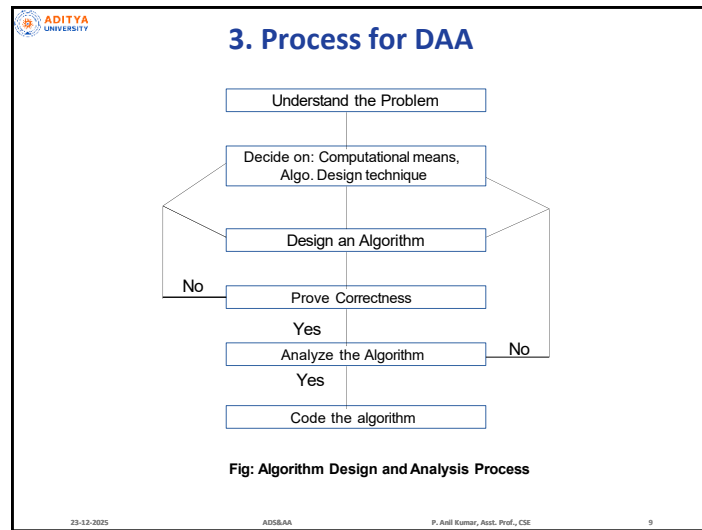
2. Characteristics / Properties

An algorithm is a **finite** sequence of **unambiguous** instructions (steps) for **solving** a problem. i.e. for obtaining a required **output** for any legitimate (valid) **input** in a **finite amount** of time.

- Finiteness
- Definiteness
- Output
- Input
- Effectiveness
- Correctness

Criteria for Algorithms
<i>Input:</i> Zero or more inputs
<i>Output:</i> At least one output.
<i>Finiteness:</i> N number of steps.
<i>Definiteness:</i> Clear algorithm step.
<i>Effectiveness:</i> A carried out step.

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 8



4. Performance Analysis of Algorithm

Q: Need for analysis?

- To determine resource consumption
 - CPU time
 - Memory space
- Compare different methods for solving the same problem before actually implementing them and running the programs.
- To find an efficient algorithm

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 10

5. Performance Measurement (Complexity)

- A measure of the performance of an algorithm
- Algorithm's performance depends on
 - *internal* factors
 - *external* factors

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 11

External Factor

- Speed of the computer on which it is run
- Quality of the compiler

Internal Factor

- The algorithm's efficiency, in terms of:
 - Time required to run
 - Space (memory storage) required to run

Note:
Complexity measures the *internal* factors (usually more interested in time than space)

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 12



Space Complexity

- The space needed by an algorithm is the sum of a fixed part and a variable part.
 - Space complexity $S(P) = C + S_p$ (Instance characteristics)**
- The fixed part includes space for
 - Instructions
 - Simple variables
 - Fixed size component variables, etc..
- The variable part includes space for
 - Component variables whose size is dependant on the particular problem instance being solved
 - Recursion stack space, etc..

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
13



Examples

1) Algorithm sum (int x, int y)

```

{
    int sum;
    sum = x+y;
    Return (sum);
}
        
```

2) Algorithm sum (int x, int y)

```

{
    Return (x+y);
}
        
```

3) Algorithm fun (float x, float y, float z)

```

{
    Return (X + Y +Y * Z + (X +
        Y +Z)) / (X+ Y) + 4.0;
}
        
```

4) Algorithm sumarr (int a[], int n)

```

{
    int *b,i;
    b=(int*)malloc(n*sizeof(int));
    for(i =0;i<n;i++)
    {
        b[i] = a[i];
    }
}
        
```

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
14



Examples

Function: factorial(n)
Returns: Factorial of the number n

Method:
result = 1
for i = (1..n)
{
 result = result * i
}
return result

Function: factorialRecursive(n)
Returns: Factorial of the number n

Method:
if n==0
 return 1
result = factorialRecursive(n-1)*n
return result

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
15



Examples

5) Algorithm ADD (float X[], int n)

```

{
    sum := 0.0;
    for i:=1 to n do
    {
        sum:=sum + X[i];
    }
    return sum;
}
        
```

6) Algorithm COPY2D (float a[][], int n)

```

{
    int i,j,b[][10];
    for i:=1 to r do
    {
        for j:=1 to c do
        {
            b[i][j] = a[i][j]
        }
    }
    return b;
}
        
```

7)

```

1. int f (int n)
2. {
3.     if (n >= 1)
4.     {
5.         Display (n);
6.         f (n-1);
7.     }
8. }

9. int main()
10. {
11.     f (2);
12. }
        
```

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
16

```

f(2)
{
  if (2 >= 1) //TRUE
  {
    Display (2);
    f(2-1);
  }
}

f(1)
{
  if (1 >= 1) //TRUE
  {
    Display (1);
    f(1-1);
  }
}

f(0)
{
  if (0 >= 1) //FALSE
  {
    // not consider
  }
}

```

Parameters
 Local Variables
 Return Address

Fig: Activation Record

n = 0 Return Address (Line #5)
n = 1 Return Address (Line #5)
n = 2 Return Address (Line #11)

Output: 1 2

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
17

Time Complexity

- The time complexity of a problem is
 - The number of steps that it takes to solve an instance of the problem as a function of the size of the input (usually measured in bits), using the most efficient algorithm.
 - Priori analysis or *compile time*
 - Posteriori analysis or *run (execution) time*.
- Time complexity $T(P) = C + T(n)$**
 - It can be two measured in two ways:
 1. By using Global count variable
 2. By using Tabular Method

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
18

ADITYA UNIVERSITY
Contd ...

1. Global variable count method:(Operation Count)

```

Algorithm Sum(a, n)
{
  s:=0;
  for i:=1 to n do
  {
    s:=s+a[i];
  }
  return s;
}

```

```

count:=0;
Algorithm Sum(a,n)
{
  s:=0;
  count:=count+1;
  for i:=1 to n do
  {
    count:=count +1;
    s:=s+a[i];
    count:=count+1;
  }
  count:=count+1; //for last time of
  return s;
  count:=count+1; //for return
}

```

Thus the total number of steps are $2n+3$

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
19

ADITYA UNIVERSITY
Contd ...

2. Tabular method (Step Count):

Ex:

Statement	S/e	Frequency	Total steps
1. Algorithm Sum(a, n)	0	-	0
2. {	0	-	0
3. s:=0;	1	1	1
4. for i:=1 to n do	1	n+1	n+1
5. s:=s+a[i];	1	n	n
6. return s;	1	1	1
7. }	0	-	0
Total			2n+3 steps

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
20

Time complexity cases

- **Best Case:** Inputs are provided in such a way that the minimum time is required to process them.
- **Average Case:** The amount of time the algorithm takes on an average set of inputs.
- **Worst Case:** The amount of time the algorithm takes on the worst possible set of inputs.

Example: Linear Search

3	4	5	6	7	9	10	12	15
---	---	---	---	---	---	----	----	----

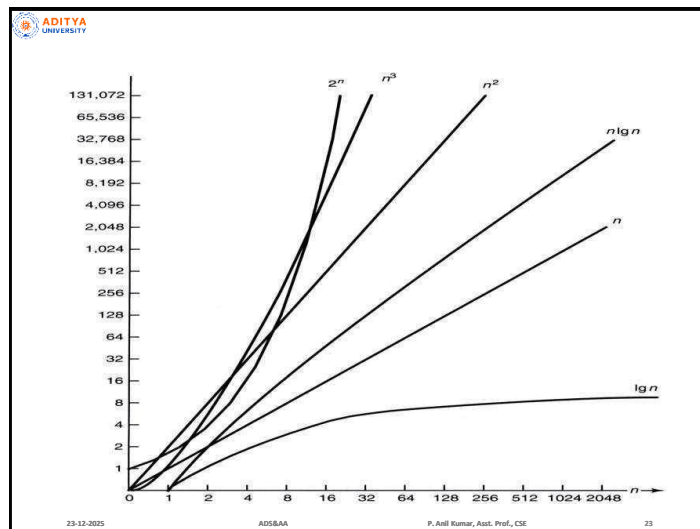
A 1 2 3 4 5 6 7 8 9

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 21

Common growth rates

Time complexity	Example
$O(1)$ <i>constant</i>	Adding to the front of a linked list
$O(\log N)$ <i>log</i>	Finding an entry in a sorted array
$O(N)$ <i>linear</i>	Finding an entry in an unsorted array
$O(N \log N)$ <i>n-log-n</i>	Sorting n items by 'divide-and-conquer'
$O(N^2)$ <i>quadratic</i>	Shortest path between two nodes in a graph
$O(N^3)$ <i>cubic</i>	Simultaneous linear equations
$O(2^N)$ <i>exponential</i>	The Towers of Hanoi problem

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 22



6. Asymptotic Notation

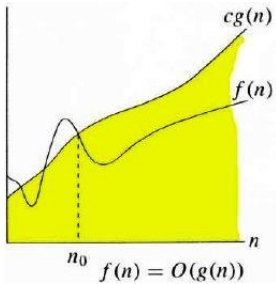
- The exact number of steps will depend on exactly what machine or language is being used.
- To avoid that problem, the Asymptotic notation is generally used.
- Running time of an algorithm as a function of input size n for large data n.
- Expressed using only the highest-order term in the expression for the exact running time.

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 24

Big - Oh notation: O

Let $f(n)$ and $g(n)$ be the two non-negative functions. We say that $f(n)$ is said to be $O(g(n))$ if and only if there exists a positive constant ' c ' and ' n_0 ' such that, $f(n) \leq c \cdot g(n)$ for all non-negative values of n , where $n \geq n_0$.

Here, $g(n)$ is the upper bound for $f(n)$.



$f(n) = O(g(n))$

Find upper bound of running time of a linear function $f(n) = 6n + 3$.

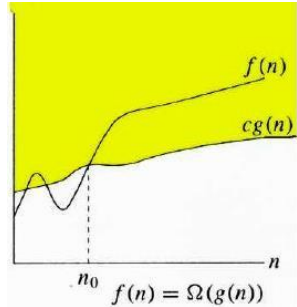
$f(n) = 3n^2 + 2n + 4$.

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 25

Big - Omega notation: Ω

Let $f(n)$ and $g(n)$ be the two non-negative functions. We say that $f(n)$ is said to be $\Omega(g(n))$ if and only if there exists a positive constant ' c ' and ' n_0 ' such that, $f(n) \geq c \cdot g(n)$ for all non-negative values of n , where $n \geq n_0$.

Here, $g(n)$ is the lower bound for $f(n)$.



$f(n) = \Omega(g(n))$

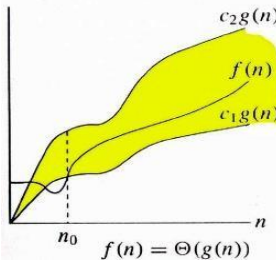
Find lower bound of running time of a linear function $f(n) = 6n + 3$.

$f(n) = 3n^2 + 2n + 4$.

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 26

Theta notation: Θ

Let $f(n)$ and $g(n)$ be the two non-negative functions. We say that $f(n)$ is said to be $\Theta(g(n))$ if and only if there exists a positive constants ' c_1 ' and ' c_2 ', such that, $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all non-negative values n , where $n \geq n_0$.



$f(n) = \Theta(g(n))$

The above definition states that the function $f(n)$ lies between ' c_1 ' times the function $g(n)$ and ' c_2 ', times the function $g(n)$ where ' c_1 ' and ' c_2 ' are positive constants.

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 27

Show that the following equalities are incorrect with suitable notations.

i) $10n^2 + 9 = O(n)$ (Big-Oh)
 ii) $n^2 \log n = \Theta(n^2)$ (Theta)

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 31



Big-Oh vs Big-Omega

- Big oh notation is denoted by 'O', whereas Omega notation is denoted by 'Ω'.
- Big oh is used to represent the upper bound of an algorithm's running time, i.e. we can give largest amount of time taken by the algorithm to complete. But whereas omega notation represent the lower bound of an algorithm's running time, i.e. we can give smallest amount of time taken by the algorithm to complete.
- Let $f(n)$ and $g(n)$ be the two non-negative functions. We say that $f(n)$ is said to be $O(g(n))$ if and only if there exists a positive constant 'c' and 'n0' such that, $f(n) \leq c * g(n)$ for all non-negative values of n, where $n \geq n_0$. But whereas function $f(n) = \Omega(g(n))$ (read as for of n is omega of g of n) if and only if there exist positive constants 'c' and 'n0' such that, $f(n) \geq c * g(n)$ for all n, $n \geq n_0$.

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
29



Asymptotic Notation Properties

- General property
- Transitive
- Reflexive
- Symmetry
- Transpose Symmetry

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
30



General property

- If $f(n)$ is $O(g(n))$ then $a * f(n)$ is $O(g(n))$
- If $f(n)$ is $\Theta(g(n))$ then $a * f(n)$ is $\Theta(g(n))$
- If $f(n)$ is $\Omega(g(n))$ then $a * f(n)$ is $\Omega(g(n))$ for any 'a'

Eg: $f(n) = 2n^2 + 5$ is $O(n^2)$.
Let as assume $a = 7$

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
31



Transitive

"Given three functions, f, g and h, we expect that ordering is transitive: if $h \leq g$ and $g \leq f$ then $h \leq f$ "

- If $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$, then $f(n) = \Theta(h(n))$
- If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$
- If $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$, then $f(n) = \Omega(h(n))$

Eg: $f(n) = n$, $g(n) = n^2$, $h(n) = n^3$

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
32



Reflexivity

“The reflexive property states that any real number, a , is equal to itself. That is, $a = a$.”

- $f(n) = \Theta(f(n))$
- $f(n) = O(f(n))$
- $f(n) = \Omega(f(n))$

Eg: $f(n) = n^2$ is $O(n^2)$

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 33

Symmetry

“This property states that if $a = b$, then $b = a$ ”

- $f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$
- Eg: $f(n) = n^2$, $g(n) = n^2$

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 34



Transpose Symmetry

- $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$

Eg: $f(n) = n$, $g(n) = n^2$

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 35



Hashing

- The *hash table* ADT which supports only a subset of the operations allowed by binary search trees.
- The implementation of hash tables is called **hashing**.
- Hashing is a technique used for performing insertions, deletions and finds in constant average time (i.e. $O(1)$)
- This data structure, however, is not efficient in operations that require any ordering information among the elements, such as findMin, findMax and printing the entire table in sorted order.

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 36



General Idea

- The ideal hash table structure is merely an array of some fixed size, containing the items.
- A stored item needs to have a data member, called **key**, that will be used in computing the index value for the item.
 - Key could be an *integer*, a *string*, etc
 - e.g. a name or Id that is a part of a large employee structure
- The size of the array is *TableSize*.
- The items that are stored in the hash table are indexed by values from 0 to *TableSize* – 1.
- Each key is mapped into some number in the range 0 to *TableSize* – 1.
- The mapping is called a *hash function*.

23-12-2025

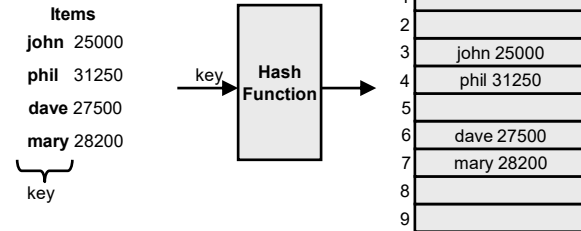
ADS&AA

P. Anil Kumar, Asst. Prof., CSE

37



Example



23-12-2025

ADS&AA

P. Anil Kumar, Asst. Prof., CSE

38



Hash Function

- The hash function:
 - must be simple to compute.
 - must distribute the keys evenly among the cells.
- If we know which keys will occur in advance we can write *perfect* hash functions, but we don't.

23-12-2025

ADS&AA

P. Anil Kumar, Asst. Prof., CSE

39



Contd ...

Problems:

- Keys may not be numeric.
- Number of possible keys is much larger than the space available in table.
- Different keys may map into same location
 - Hash function is not one-to-one => collision.
 - If there are too many collisions, the performance of the hash table will suffer dramatically.

23-12-2025

ADS&AA

P. Anil Kumar, Asst. Prof., CSE

40



Hash Functions

- If the input keys are integers then simply $Key \bmod TableSize$ is a general strategy.
 - Unless key happens to have some undesirable properties. (e.g. all keys end in 0 and we use mod 10)
- If the keys are strings, hash function needs more care.
 - First convert it into a numeric value.

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
41



Some methods

- **Truncation:**
 - e.g. 123456789 map to a table of 1000 addresses by picking 3 digits of the key.
- **Folding:**
 - e.g. 123|456|789: add them and take mod.
- **Key mod N:**
 - N is the size of the table, better if it is prime.
- **Squaring:**
 - Square the key and then truncate
- **Radix conversion:**
 - e.g. 1 2 3 4 treat it to be base 11, truncate if necessary.

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
42



Hash function for strings

$97 \quad 108 \quad 105 \rightarrow key[i]$
 $\uparrow \quad \uparrow \quad \uparrow$
 $key \quad \boxed{a} \quad \boxed{l} \quad \boxed{i}$
 $0 \quad 1 \quad 2 \rightarrow i$
 $KeySize = 3;$

$hash("ali") = (105 * 1 + 108 * 37 + 97 * 37^2) \% 10,007 = 6803$

"ali" → hash function →
 0
1
2
.....
6803
ali
.....
10,006 (TableSize)

Rabin-Karp Algorithm

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
43



Collision Resolution

- If, when an element is inserted, it hashes to the same value as an already inserted element, then we have a collision and need to resolve it.
- There are several methods for dealing with this:
 - **Separate chaining (open hashing)**
 - **Open addressing (closed hashing)**
 - Linear Probing
 - Quadratic Probing
 - Double Hashing

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
44



Separate Chaining

- The idea is to keep a list of all elements that hash to the same value.
 - The array elements are pointers to the first nodes of the lists.
 - A new item is inserted to the front of the list.
- Advantages:
 - Better space utilization for large items.
 - Simple collision handling: searching linked list.
 - Overflow: we can store more items than the hash table size.
 - Deletion is quick and easy: deletion from the linked list.

23-12-2025

ADS&AA

P. Anil Kumar, Asst. Prof., CSE

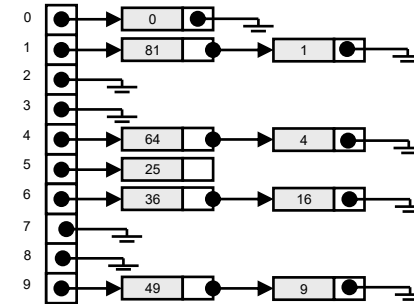
45



Example

Keys: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81

hash(key) = key % 10.



23-12-2025

ADS&AA

P. Anil Kumar, Asst. Prof., CSE

46



Operations

- Initialization:** all entries are set to NULL
- Find:**
 - locate the cell using hash function.
 - sequential search on the linked list in that cell.
- Insertion:**
 - Locate the cell using hash function.
 - (If the item does not exist) insert it as the first item in the list.
- Deletion:**
 - Locate the cell using hash function.
 - Delete the item from the linked list.

23-12-2025

ADS&AA

P. Anil Kumar, Asst. Prof., CSE

47



Analysis of Separate Chaining

- Collisions are very likely.
 - How likely and what is the average length of lists?
- Load factor λ definition:
 - Ratio of number of elements (N) in a hash table to the hash *TableSize*.
 - i.e. $\lambda = N/TableSize$
 - The average length of a list is also λ .
 - For chaining λ is not bound by 1; it can be > 1 .

23-12-2025

ADS&AA

P. Anil Kumar, Asst. Prof., CSE

48

 **ADITYA UNIVERSITY**

Cost of searching

- **Cost** = Constant time to evaluate the hash function + time to traverse the list.
- **Unsuccessful search:**
 - We have to traverse the entire list, so we need to compare λ nodes on the average.
- **Successful search:**
 - List contains the one node that stores the searched item + 0 or more other nodes.
 - Expected # of other nodes = $x = (N-1)/M$ which is essentially λ , since M is presumed large.
 - On the average, we need to check *half* of the *other nodes* while searching for a certain element
 - Thus average search cost = $1 + \lambda/2$

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 49

 **ADITYA UNIVERSITY**

Collision Resolution with Open Addressing

- Separate chaining has the disadvantage of using linked lists.
 - Requires the implementation of a second data structure.
- In an open addressing hashing system, all the data go inside the table.
 - Thus, a bigger table is needed.
 - Generally the load factor should be below 0.5.
 - If a collision occurs, alternative cells are tried until an empty cell is found.

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 50

 **ADITYA UNIVERSITY**

Open Addressing

- More formally:
 - Cells $h_0(x)$, $h_1(x)$, $h_2(x)$, ... are tried in succession where $h_i(x) = (\text{hash}(x) + f(i)) \bmod \text{TableSize}$, with $f(0) = 0$.
 - The function f is the collision resolution strategy.
- There are three common collision resolution strategies:
 - Linear Probing
 - Quadratic probing
 - Double hashing

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 51

 **ADITYA UNIVERSITY**

Linear Probing

- In linear probing, collisions are resolved by sequentially scanning an array (with wraparound) until an empty cell is found.
 - i.e. f is a linear function of i , typically $f(i) = i$.
- Example:
 - Insert items with keys: 89, 18, 49, 58, 9 into an empty hash table.
 - Table size is 10.
 - Hash function is $\text{hash}(x) = x \bmod 10$.
 - $f(i) = i$;

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 52

ADITYA UNIVERSITY

hash (89, 10) = 9
 hash (18, 10) = 8
 hash (49, 10) = 9
 hash (58, 10) = 8
 hash (9, 10) = 9

Linear probing hash table after each insertion

	After insert 89	After insert 18	After insert 49	After insert 58	After insert 9
0			49	49	49
1				58	58
2					9
3					
4					
5					
6					
7					
8		18	18	18	18
9	89	89	89	89	89

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 53

ADITYA UNIVERSITY

Find and Delete

- The find algorithm follows the same probe sequence as the insert algorithm.
 - A find for 58 would involve 4 probes.
 - A find for 19 would involve 5 probes.
- We must use *lazy deletion* (i.e. marking items as deleted)
 - Standard deletion (i.e. physically removing the item) cannot be performed.
 - e.g. remove 89 from hash table.

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 54

ADITYA UNIVERSITY

Clustering Problem

- As long as table is big enough, a free cell can always be found, but the time to do so can get quite large.
- Worse, even if the table is relatively empty, blocks of occupied cells start forming.
- This effect is known as *primary clustering*.
- Any key that hashes into the cluster will require several attempts to resolve the collision, and then it will add to the cluster.

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 55

ADITYA UNIVERSITY

Linear Probing - Example

What is the average number of probes for a successful search and an unsuccessful search for this hash table?

– Hash Function: $h(x) = x \text{ mod } 11$

Successful Search:

– 20: 9 -- 30: 8 -- 2: 2 -- 13: 2, 3 -- 25: 3, 4
 – 24: 2, 3, 4, 5 -- 10: 10 -- 9: 9, 10, 0
Avg. Probe for SS = $(1+1+1+2+2+4+1+3)/8=15/8$

Unsuccessful Search:

– We assume that the hash function uniformly distributes the keys.
 – 0: 0, 1 -- 1: 1 -- 2: 2, 3, 4, 5, 6 -- 3: 3, 4, 5, 6
 – 4: 4, 5, 6 -- 5: 5, 6 -- 6: 6 -- 7: 7 -- 8: 8, 9, 10, 0, 1
 – 9: 9, 10, 0, 1 -- 10: 10, 0, 1
Avg. Probe for US = $(2+1+5+4+3+2+1+1+5+4+3)/11=31/11$

0	9
1	
2	2
3	13
4	25
5	24
6	
7	
8	30
9	20
10	10

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 56



Quadratic Probing

- Quadratic Probing eliminates primary clustering problem of linear probing.
- Collision function is quadratic.
 - The popular choice is $f(i) = i^2$.
- If the hash function evaluates to h and a search in cell h is inconclusive, we try cells $h + 1^2$, $h + 2^2$, ... $h + i^2$.
 - i.e. It examines cells 1,4,9 and so on away from the original probe.
- Remember that subsequent probe points are a quadratic number of positions from the *original probe point*.
- Ex: 11, 9, 22, 33, 44, 55, 66.

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
57



Quadratic Probing

hash (89, 10) = 9
 hash (18, 10) = 8
 hash (49, 10) = 9
 hash (58, 10) = 8
 hash (9, 10) = 9

After insert 89 After insert 18 After insert 49 After insert 58 After insert 9

0			49	49	49
1					
2				58	58
3					9
4					
5					
6					
7					
8		18	18	18	18
9	89	89	89	89	89

A quadratic probing hash table after each insertion (note that the table size was poorly chosen because it is not a prime number).

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
58



Quadratic Probing

- Problem:
 - We may not be sure that we will probe all locations in the table (i.e. there is no guarantee to find an empty cell if table is more than half full.)
 - If the hash table size is not prime this problem will be much severe.
- However, there is a theorem stating that:
 - If the table size is *prime* and load factor is not larger than 0.5, all probes will be to different locations and an item can always be inserted.

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
59



Some considerations

- How efficient is calculating the quadratic probes?
 - Linear probing is easily implemented. Quadratic probing appears to require * and % operations.
 - However by the use of the following trick, this is overcome:
 - $H_i = H_{i-1} + 2i - 1 \pmod{M}$

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
60



Some Considerations

- What happens if load factor gets too high?
 - Dynamically expand the table as soon as the load factor reaches 0.5, which is called *rehashing*.
 - Always double to a prime number.
 - When expanding the hash table, reinsert the new table by using the new hash function.

23-12-2025

ADS&AA

P. Anil Kumar, Asst. Prof., CSE

61



Analysis of Quadratic Probing

- Quadratic probing has not yet been mathematically analyzed.
- Although quadratic probing eliminates primary clustering, elements that hash to the same location will probe the same alternative cells. This is known as *secondary clustering*.
- Techniques that eliminate secondary clustering are available.
 - the most popular is *double hashing*.

23-12-2025

ADS&AA

P. Anil Kumar, Asst. Prof., CSE

62



Double Hashing

- A second hash function is used to drive the collision resolution.
 - $f(i) = i * hash_2(x)$
- We apply a second hash function to x and probe at a distance $hash_2(x)$, $2 * hash_2(x)$, ... and so on.
- The function $hash_2(x)$ must never evaluate to zero.
 - e.g. Let $hash_2(x) = x \bmod 9$ and try to insert 99 in the previous example.
- A function such as $hash_2(x) = R - (x \bmod R)$ with R a prime smaller than TableSize will work well.
 - e.g. try $R = 7$ for the previous example. $(7 - x \bmod 7)$

23-12-2025

ADS&AA

P. Anil Kumar, Asst. Prof., CSE

63



Need for Dynamic Hashing

- Hashing is a technique to map keys to memory locations.
- Static hashing uses a fixed table size.
- Static hashing causes bucket overflow.
- Long overflow chains reduce efficiency.
- Rehashing entire table is costly.
- Dynamic hashing solves these problems.
- Dynamic hashing allows table size to grow or shrink.
- Improves performance when data size changes.

23-12-2025

ADS&AA

P. Anil Kumar, Asst. Prof., CSE

64



Dynamic Hashing

- A hashing technique where hash table expands dynamically.
- Buckets are split when overflow occurs.
- Maintains fast insertion, deletion, and searching.

Types of Dynamic Hashing

- Extendible Hashing
- Linear Hashing

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
65



Extendible Hashing - Terms

- **Hash Function:** Converts keys into binary values
- **Directory:** Array of pointers to buckets
- **Global Depth (GD):** Number of bits used to index the directory
- **Local Depth (LD):** Number of bits used to identify a bucket
- **Bucket Capacity:** Maximum keys a bucket can store

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
66



Extendible Hashing – Concept

- Directory size = 2^{GD}
- Each directory entry points to a bucket
- Multiple directory entries can point to the same bucket
- Buckets split when overflow occurs

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
67



Working Principle

1. Compute hash value of key
2. Use last **GD bits** to index directory
3. Insert key into the corresponding bucket
4. If bucket overflows:
 - Split bucket
 - Increase local depth
 - Double directory size if needed
 - Redistribute keys

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
68

ADITYA UNIVERSITY **Example of Extendible Hashing**

Global Depth (GD) = 1
 So the directory has:
 $2^{GD} = 2^1 = 2$ entries
 Directory (using last 1 bit of hash value):

Directory Entry	Points to Bucket
0	B0
1	B1

Each bucket:
 Local Depth (LD) = 1
 Capacity = 2

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 69

ADITYA UNIVERSITY **Example of Extendible Hashing**

Insert Keys

Key	Hash Value
5	101
7	111
3	011
1	001

Insert Key 5 (101)
 Last 1 bit → 1
 Goes to **Bucket B1**
 B1 = {5}

Insert Key 7 (111)
 Last 1 bit → 1
 Goes to **Bucket B1**
 B1 = {5, 7} (bucket is now full)

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 70

ADITYA UNIVERSITY **Example of Extendible Hashing**

Insert Keys

Key	Hash Value
5	101
7	111
3	011
1	001

Insert Key 3 (011)
 Last 1 bit → 1
 Goes to **Bucket B1**
 But **B1 is full (Overflow)**

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 71

ADITYA UNIVERSITY **Example of Extendible Hashing**

Insert Keys

Key	Hash Value
5	101
7	111
3	011
1	001

Bucket Split
 Since **Local Depth (1) = Global Depth (1)**

Increase Global Depth
 GD = 2
 Directory size becomes:
 $2^2 = 4$

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 72



Example of Extendible Hashing

New Directory as:

Entry	Last 2 bits
00	B0
01	B1
10	B0
11	B1

Split Bucket B1

- Create a new bucket B2
- Increase local depth of B1 and B2 to 2

Redistribute keys using **last 2 bits**:

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
73



Example of Extendible Hashing

Key	Hash	Last 2 bits	Bucket
5	101	01	B1
7	111	11	B2
3	011	11	B2

Insert Key 1 (001)
Last 2 bits → **01**

Goes to **Bucket B1**
B1 = {5, 1} (fits exactly)

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
74



Example of Extendible Hashing

Final Hash Table

Directory	Bucket	Contents
00	B0	empty
01	B1	{5, 1}
10	B0	empty
11	B2	{7, 3}

- Hash table size** in extendible hashing means **directory size**, not number of buckets.
- Buckets increase only when splitting happens, but directory size depends **only on global depth**.

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
75



Linear Hashing

Linear hashing is a **dynamic hashing technique** in which the hash table **grows gradually (one bucket at a time)** instead of doubling suddenly, as in extendible hashing.

Key Idea

- No directory is used
- Buckets are split **one at a time**
- Hash function changes **incrementally**
- Table grows **linearly**

23-12-2025
ADS&AA
P. Anil Kumar, Asst. Prof., CSE
76

ADITYA UNIVERSITY

Linear Hashing

Linear hashing is a dynamic hashing technique in which the hash table **grows gradually (one bucket at a time)** instead of doubling suddenly, as in extendible hashing.

Key Idea

1. Initial number of buckets (m).
2. Hash functions: $h_i(k) = k \bmod (m \times 2^i)$.
3. Level (i): indicates which hash function is currently in use.
4. Split pointer (s): points to the next bucket to be split.

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 77

ADITYA UNIVERSITY

Linear Hashing - Example

Initial Setup
 Number of buckets: 2
 Bucket 0 (B0)
 Bucket 1 (B1)
 Each bucket can store 2 keys
 Hash function: $h(k) = k \bmod 2$

Keys to Insert: 2, 4, 6, 8

Insert 2 and 4

Key	$h(k) = k \bmod 2$	Bucket
2	0	B0
4	0	B0

B0: [2, 4] (FULL)
B1: []

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 78

ADITYA UNIVERSITY

Linear Hashing - Example

Insert 6, Overflow Happens $h(6) = 0$
 Bucket B0 is full, so we **split B0**.
What does split mean?
 Create **one new bucket** (B2)
 Recalculate keys using a new hash: $h'(k) = k \bmod 4$
Rehash keys of B0

Key	$k \bmod 4$	New Bucket
2	2	B2
4	0	B0
6	2	B2

B0: [4]
B1: []
B2: [2, 6]

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 79

ADITYA UNIVERSITY

Linear Hashing - Example

Insert 8 $8 \bmod 4 = 0$

B0: [4, 8]
B2: [2, 6]

Final Table

B0: [4, 8]
B1: []
B2: [2, 6]

Advantages

- Table **grows slowly**.
- Only **one bucket split**.
- Old data mostly stays where it is.
- Efficient and simple.

23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 80

		
Feature	Linear Dynamic Hashing	Extendible Dynamic Hashing
Directory	Not used	Used
Growth	Gradual (one bucket at a time)	Sudden (directory doubling)
Overflow handling	Bucket splitting in order	Bucket splitting using depth
Memory overhead	Low	High (directory)
Search time	Slightly variable	Very fast and predictable
Implementation	Moderate	More complex
Space utilization	Efficient	Sometimes wasteful
Used in	Databases, file systems	Database indexing
23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 81		

			
Hashing Applications			
- Compilers use hash tables to implement the <i>symbol table</i> (a data structure to keep track of declared variables). - Game programs use hash tables to keep track of positions it has encountered (<i>transposition table</i>) - Online spelling checkers. (ex: “tec”)			
23-12-2025 ADS&AA P. Anil Kumar, Asst. Prof., CSE 82			